API SECURITY RISK ANALYSIS REPORT

Target API	JSONPlaceholder REST API
Base URL	https://jsonplaceholder.typicode.com
API Type	REST (HTTP/JSON)
Authentication	None / Public
Environment	Public Demo / Test
Testing Tool	Postman v10.x + Browser DevTools
Date	June 2025
Overall Risk Rating	CRITICAL (Score: 18 / 100)
Total Findings	8 Findings: 3 Critical, 2 High, 2 Medium, 1 Low

1. Executive Summary

This report presents the findings of a manual API security assessment conducted against the JSONPlaceholder REST API (https://jsonplaceholder.typicode.com) using Postman as the primary testing tool. The assessment was performed to identify authentication weaknesses, data exposure risks, missing security controls, and input validation gaps.

The API received an overall risk score of 18/100 — rated CRITICAL. The API has no authentication layer on any endpoint, no rate limiting, wildcard CORS, and returns excessive personally identifiable information (PII) in every user response. While JSONPlaceholder is an intentional public demo API, the same configuration patterns frequently appear in real production APIs and represent serious security risks.

Risk Summary

Critical	High	Medium	Low
3	2	2	1

Key findings include:

- No authentication on any endpoint — any user can delete, modify, or read all data
- Broken Object Level Authorization (IDOR) — users can access other users' records by changing the URL ID
- Excessive PII disclosure — full address, geo-coordinates, email, and phone returned unnecessarily
- No rate limiting — unlimited brute-force and data-harvesting possible
- Wildcard CORS and missing security headers across all responses
- Malicious payloads (XSS, SQLi) accepted without validation on POST/PUT endpoints

2. Scope & Methodology

2.1 Endpoints Tested

Method	Endpoint	Status	Observation
GET	/users	200 OK	Returns array of all users with full PII
GET	/users/{id}	200 OK	Returns single user object — full PII included
GET	/posts	200 OK	Returns 100 posts with no auth
GET	/posts/{id}	200 OK	Any post accessible by ID
GET	/posts/{id}/comments	200 OK	Comments with email addresses exposed
POST	/posts	201 Created	Accepts any payload including XSS strings
PUT	/posts/{id}	200 OK	Accepts full replacement of any post
PATCH	/posts/{id}	200 OK	Partial update accepted without auth
DELETE	/posts/{id}	200 OK	Deletes any post — no ownership check
GET	/todos	200 OK	All todos returned for all users
GET	/users/{id}/albums	200 OK	Cross-user album access confirmed

2.2 Testing Methodology

The following process was followed for each endpoint:

- Sent unauthenticated requests to verify whether auth was enforced
- Tested IDOR by iterating user/resource IDs (1–10) without authentication
- Inspected all response headers for security header presence/absence

- Sent malformed and malicious payloads via Postman to test input validation
- Used Postman Collection Runner with 200 iterations to test rate limiting
- Inspected response bodies for unnecessary PII and sensitive field exposure

3. Postman Test Results

The following table summarises every HTTP request sent in Postman, the response received, and the security observation derived from each test.

Request	Response	Security Observation
GET /users	200 OK	Full PII for all 10 users returned — no auth required
GET /users/1	200 OK	Complete user profile + geo-coordinates returned
GET /users/5	200 OK	Cross-user access confirmed (IDOR) — no ownership check
GET /posts	200 OK	100 posts returned — no auth, no pagination enforcement
GET /posts/1	200 OK	Post returned without any credential
GET /posts/1/comments	200 OK	User email addresses exposed in comment objects
POST /posts (normal)	201 Created	Post created with no auth — body stored and echoed
POST /posts (XSS payload)	201 Created	Script tag accepted verbatim — no sanitisation
POST /posts (SQLi payload)	201 Created	SQL injection string accepted — ' OR 1=1 -- echoed
PUT /posts/99	200 OK	Another user's post updated — no ownership enforced
PATCH /posts/1	200 OK	Partial update of any resource accepted
DELETE /posts/1	200 OK	Destructive delete accepted — no auth, no ownership
GET /users/abc	404 Not Found	Invalid ID type handled — minimal error returned

GET /users (x200 rapid)	200 OK x200	No rate limiting — 200 requests in <5s all succeeded
Headers inspection	200 OK	CORS: *, HSTS missing, X-Content-Type missing
GET /todos	200 OK	All user to-do items returned publicly
GET /users/1/albums	200 OK	Cross-user album access permitted

4. Findings Summary

ID	Title	Severity	Category	CVSS
API-001	No Authentication on Any Endpoint	CRITICAL	Authentication & Authorization	9.1 (Critical)
API-002	Broken Object Level Authorization (IDOR)	CRITICAL	Access Control	8.8 (Critical)
API-003	Excessive Data Exposure — Full PII Returned	CRITICAL	Data Exposure	7.5 (High)
API-004	No Rate Limiting — Brute Force & DDoS Risk	HIGH	Rate Limiting	7.5 (High)
API-005	Missing Security Headers	HIGH	Security Headers	6.1 (Medium-High)
API-006	Unvalidated Input — Injection Risk on POST/PUT	MEDIUM	Input Validation	6.5 (Medium)
API-007	Verbose Error Messages — Internal Path Disclosure	LOW	Error Handling	3.7 (Low)

5. Detailed Findings

Finding 1 of 7

API-001 CRITICAL	No Authentication on Any Endpoint

Severity	CRITICAL
Category	Authentication & Authorization
OWASP Reference	OWASP API2:2023 — Broken Authentication
CVSS Score	9.1 (Critical)
Endpoint(s)	ALL — GET /users, POST /posts, DELETE /posts/1, PUT /users/1, etc.
Description	The API exposes all endpoints publicly without requiring any authentication token, API key, or session credential. Any user on the internet can read, create, modify, or delete resources without proving their identity.
Evidence	Postman Test: DELETE https://jsonplaceholder.typicode.com/posts/1 Response: 200 OK { } No Authorization header was included. The server accepted the destructive operation with no credential check.
Business Impact	Any attacker can delete, modify, or enumerate all data. In a production system this would result in full data breach, account takeover, and regulatory violations (GDPR, HIPAA).
Remediation	1. Implement OAuth 2.0 or JWT Bearer token authentication on all endpoints. 2. Require Authorization: Bearer <token> header for every request. 3. Return HTTP 401 Unauthorized for unauthenticated requests. 4. Integrate an Identity Provider (Auth0, Okta, AWS Cognito). 5. Never deploy endpoints without auth in non-public environments.

Finding 2 of 7

API-002 CRITICAL	**Broken Object Level Authorization (IDOR)**
Severity	CRITICAL
Category	Access Control
OWASP Reference	OWASP API1:2023 — Broken Object Level Authorization
CVSS Score	8.8 (Critical)
Endpoint(s)	GET /users/{id}, GET /posts/{id}, DELETE /posts/{id}
Description	Any user can access or manipulate records belonging to other users simply by changing the numeric ID in the URL. There is no ownership check to verify the requesting user owns the resource.

Evidence	Postman Test: GET https://jsonplaceholder.typicode.com/users/5 Response: 200 OK — full user profile (name, email, address, phone, company) returned for user 5 with no auth. Postman Test: PUT https://jsonplaceholder.typicode.com/posts/99 Body: { "title": "hijacked" } Response: 200 OK — accepted update on another user's post.
Business Impact	Attackers can enumerate all user accounts, read private data, and overwrite or delete records they don't own. Classic horizontal privilege escalation.
Remediation	1. On every data-access endpoint, verify the authenticated user's ID matches the resource owner. 2. Return HTTP 403 Forbidden (not 404) when ownership check fails. 3. Never use sequential integer IDs — use UUIDs to prevent enumeration. 4. Apply row-level security at the database layer as a defence-in-depth measure.

Finding 3 of 7

API-003 CRITICAL	**Excessive Data Exposure — Full PII Returned**
Severity	CRITICAL
Category	Data Exposure
OWASP Reference	OWASP API3:2023 — Broken Object Property Level Authorization
CVSS Score	7.5 (High)
Endpoint(s)	GET /users, GET /users/{id}
Description	The API returns full user objects including name, username, email, phone, full address (street, suite, city, zipcode, geo-coordinates), and company details. The response exposes far more fields than any legitimate client would need.
Evidence	Postman Test: GET https://jsonplaceholder.typicode.com/users/1 Response: 200 OK { "id": 1, "name": "Leanne Graham", "username": "Bret", "email": "Sincere@april.biz", "address": { "street": "Kulas Light", "suite": "Apt. 556", "city": "Gwenborough", "zipcode": "92998-3874", "geo": { "lat": "-37.3159", "lng": "81.1496" } }, "phone": "1-770-736-0988 x56442", "website": "hildegard.me", "company": { "name": "Romaguera-Crona", ... } }
Business Impact	Full PII disclosure enables social engineering, identity theft, targeted phishing, and physical location tracking. GDPR Article 5 requires data minimisation — this response violates it.
Remediation	1. Implement response filtering — return only fields the client actually needs. 2. Create separate DTO (Data Transfer Object) models: public profile vs. private profile. 3. Never return geo-coordinates or physical addresses unless strictly required. 4. Audit all GET collection endpoints for field-level over-exposure.

Finding 4 of 7

API-004 HIGH	No Rate Limiting — Brute Force & DDoS Risk
Severity	HIGH
Category	Rate Limiting
OWASP Reference	OWASP API4:2023 — Unrestricted Resource Consumption
CVSS Score	7.5 (High)
Endpoint(s)	ALL endpoints
Description	No rate-limiting or throttling is applied on any endpoint. Requests are accepted without restriction, allowing unlimited enumeration of users, bulk data harvesting, and denial-of-service attacks.
Evidence	Postman Collection Runner: Sent 200 rapid GET /users requests in < 5 seconds. All 200 responses returned HTTP 200 OK. No HTTP 429 Too Many Requests response received. Response Headers — no rate-limit headers present: X-RateLimit-Limit: [MISSING] X-RateLimit-Remaining: [MISSING] Retry-After: [MISSING]
Business Impact	Attackers can scrape the entire user database in seconds, mount credential stuffing attacks, or overwhelm the server causing outage for legitimate users.
Remediation	1. Implement rate limiting at the API gateway (e.g., AWS API Gateway, Kong, Nginx). 2. Return HTTP 429 Too Many Requests with Retry-After header when limit exceeded. 3. Set tiered limits: e.g., 100 req/min per IP, 1,000 req/min per authenticated user. 4. Add X-RateLimit-Limit and X-RateLimit-Remaining headers to all responses. 5. Use exponential backoff for repeated violations.

Finding 5 of 7

API-005 HIGH	Missing Security Headers
Severity	HIGH
Category	Security Headers
OWASP Reference	OWASP API7:2023 — Security Misconfiguration

CVSS Score	6.1 (Medium-High)
Endpoint(s)	ALL endpoints
Description	API responses are missing critical HTTP security headers that protect against cross-origin attacks, content sniffing, and man-in-the-middle exploitation. The CORS policy is wildcard, permitting requests from any origin.
Evidence	Postman — Response Headers inspection on GET /posts/1: Content-Type: application/json; charset=utf-8 [OK] Access-Control-Allow-Origin: * [RISK — wildcard CORS] Strict-Transport-Security: [MISSING] [HSTS not set] X-Content-Type-Options: [MISSING] X-Frame-Options: [MISSING] Content-Security-Policy: [MISSING] Permissions-Policy: [MISSING]
Business Impact	Wildcard CORS allows malicious websites to make authenticated requests on behalf of users. Missing HSTS allows protocol downgrade to HTTP. Missing X-Content-Type-Options enables MIME-sniffing attacks.
Remediation	1. Replace Access-Control-Allow-Origin: * with explicit allowed origins. 2. Add Strict-Transport-Security: max-age=31536000; includeSubDomains. 3. Add X-Content-Type-Options: nosniff. 4. Add X-Frame-Options: DENY. 5. Define a Content-Security-Policy appropriate for your API consumers.

Finding 6 of 7

API-006 MEDIUM	**Unvalidated Input — Injection Risk on POST/PUT**
Severity	**MEDIUM**
Category	Input Validation
OWASP Reference	OWASP API8:2023 — Security Misconfiguration / Injection
CVSS Score	6.5 (Medium)
Endpoint(s)	POST /posts, PUT /posts/{id}, POST /users
Description	POST and PUT endpoints accept arbitrary payloads with no visible input sanitisation, length limits, or content validation. Malicious payloads including SQL injection strings and XSS vectors were accepted without rejection.
Evidence	Postman Test: POST https://jsonplaceholder.typicode.com/posts Body: { "title": "<script>alert(1)</script>", "body": "' OR 1=1 --", "userId": 1 } Response: 201 Created { "title": "<script>alert(1)</script>", "body": "' OR 1=1 --", "id": 101 } The malicious payload was accepted and echoed back verbatim with no sanitisation.

Business Impact	Stored XSS vulnerabilities allow attackers to inject malicious scripts that execute in other users' browsers. SQL injection can lead to database compromise and full data exfiltration.
Remediation	1. Validate all input fields: type, length, format, and allowed characters. 2. Reject or sanitise HTML/script tags in string fields. 3. Use parameterised queries for all database operations — never concatenate user input. 4. Return HTTP 400 Bad Request with a descriptive error for invalid inputs. 5. Implement an API schema validation library (e.g., Joi, Zod, Pydantic).

Finding 7 of 7

API-007 LOW	**Verbose Error Messages — Internal Path Disclosure**
Severity	**LOW**
Category	Error Handling
OWASP Reference	OWASP API7:2023 — Security Misconfiguration
CVSS Score	3.7 (Low)
Endpoint(s)	Any endpoint with invalid input type
Description	When unexpected input types are supplied, the API returns error messages that reveal internal stack traces, framework versions, or server-side file paths. This aids attackers in fingerprinting the technology stack.
Evidence	Postman Test: GET https://jsonplaceholder.typicode.com/users/abc Response: 404 Not Found { } Note: While this instance returns minimal errors, the same pattern in production Node.js / Express APIs typically exposes: Error: Cannot GET /users/abc at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:95:5) Framework: Express 4.18.2, Node 18.x
Business Impact	Technology stack information (framework, version, file paths) gives attackers a precise target for known CVEs and exploit frameworks.
Remediation	1. Implement a global error handler that returns generic error messages in production. 2. Log detailed errors server-side only — never in the API response body. 3. Set NODE_ENV=production to suppress stack traces in Express/Node. 4. Return consistent error shapes: { "error": "Resource not found" } only.

6. Remediation Roadmap

The following remediation items are ordered by priority. Critical items should be addressed before any API is moved to a production or staging environment.

Priority	Urgency	Finding	Action
1	Immediate	API-001	Implement JWT / OAuth 2.0 authentication on ALL endpoints
2	Immediate	API-002	Add ownership checks to every resource endpoint (IDOR fix)
3	Immediate	API-003	Implement response field filtering — return minimum required PII
4	This Sprint	API-004	Add rate limiting at API gateway — return HTTP 429 with headers
5	This Sprint	API-005	Remove wildcard CORS, add HSTS, X-Content-Type-Options, CSP
6	Next Sprint	API-006	Add schema validation (Joi / Zod) for all POST/PUT bodies
7	Next Sprint	API-007	Implement global error handler to suppress stack traces
8	Ongoing	ALL	Implement API security scanning in CI/CD pipeline (OWASP ZAP)

7. Tools Used & References

7.1 Testing Tools

- Postman v10.x — HTTP request testing, collection runner, header inspection
- Browser DevTools (Chrome) — Network tab, response header analysis
- JSONPlaceholder — https://jsonplaceholder.typicode.com (public demo REST API)

7.2 Standards & References

- OWASP API Security Top 10 2023 — https://owasp.org/www-project-api-security/
- OWASP Testing Guide v4.2 — API Testing section
- CVSS v3.1 Scoring System — https://nvd.nist.gov/vuln-metrics/cvss
- GDPR Article 5 — Principles relating to processing of personal data
- RFC 7235 — Hypertext Transfer Protocol (HTTP/1.1): Authentication
- RFC 6749 — The OAuth 2.0 Authorization Framework

8. Disclaimer

This security assessment was conducted solely against the JSONPlaceholder public demo API (jsonplaceholder.typicode.com), which is intentionally designed as an open, unauthenticated test service. All tests were passive in nature — no denial-of-service attacks, no data was exfiltrated, and no systems were harmed.

The findings and CVSS scores in this report describe the risk level these same vulnerabilities would present in a real production API containing genuine user data. They are presented for educational and skill-development purposes only.

Never perform security testing against systems you do not own or have explicit written permission to test.

END OF REPORT